DEVELOPER REFERENCE DOCUMENT

Akshansh Yadav Dev Explain

Everything in the resume — explained in full technical depth

A complete walkthrough of every project, every technology, and every core concept on the resume. Covers architecture decisions, how things work under the hood, and exactly where each tool was applied. Written as a teaching reference for technical interviews, portfolio reviews, and self-study.

CONTENTS

Akshansh Yadav

Full-Stack & Web3 Developer · B.Tech, HBTU Kanpur (2025–2029)

250102005@HBTU.AC.IN · akshanshyadav26220.dpskln@gmail.com

github.com/expediator · expediator.github.io/resume · x.com/expediator_

June 2026

01

About & Background

Akshansh Yadav is a self-taught developer pursuing a B.Tech at Harcourt Butler Technical University (HBTU), Kanpur (2025–2029). He started building before college, driven by curiosity across full-stack web, computer vision, and embedded hardware. Every project listed is shipped — not a toy demo.

TIMELINE

- **2021** — Co-founded Simbha Kunj Foundation (animal welfare + student support NGO); Social Media Lead
- **2022** — Class X: 93.6% overall
- **2023–24** — Independent E-Commerce & Dropshipping: product sourcing, digital marketing funnels, data-driven sales analysis
- **2024** — Class XII: 85%; Qualified JEE Mains 2024 + JEE Advanced 2024
- **2025** — Qualified JEE Mains 2025 + JEE Advanced 2025; built Space Shooter + RC Car; joined HBTU Kanpur
- **2025** — SOF Olympiads (Class 11): English Intl. Rank 240; RC Plane design, assembly & flight testing
- **Oct 2025** — Social & Outreach Head, YouTube Channel
- **2026** — SwapX (CEX+DEX crypto exchange), FaceMetrics, Interactive Portfolio — all shipped

WHAT HE BUILDS

- **Full-Stack Web** — production Next.js apps with auth, real-time data, cloud deployment (SwapX)
- **Computer Vision & Embedded** — browser + Python CV tools + hardware interfacing (FaceMetrics, RC Car)
- **Web3 (Exploring)** — DeFi mechanics, AMM/order-book design, Solidity, Anchor Framework

LINKS

Platform	Link
GitHub	github.com/expediator
Portfolio	expediator.github.io/resume
X (Twitter)	x.com/expediator_
LinkedIn	linkedin.com/in/akshansh-yadav-554926304
Email (HBTU)	250102005@HBTU.AC.IN
Email (personal)	akshanshyadav26220.dpskln@gmail.com
Resume PDF	expediator.github.io/resume/assets/Akshansh-Yadav-Resume.pdf

EDUCATION

Qualification	Institution	Year	Result
B.Tech	HBTU Kanpur (Harcourt Butler Technical University)	2025–2029	In progress
Class XII	DPS RK Puram (CBSE)	2024	85% overall
Class X	DPS RK Puram (CBSE)	2022	93.6% overall

ACHIEVEMENTS

- Qualified JEE Mains 2024 & 2025; JEE Advanced 2024 & 2025
- SOF Olympiads (Class 11) — English: Intl. Rank 240 · Science: 1084 · Maths: 3872
- RC Plane Development — designed, assembled, and flight-tested a fixed-wing RC aircraft from scratch

- Co-Founded Simbha Kunj Foundation (NGO) — animal welfare & student support, Kanpur

02

Projects — Deep Technical Breakdown

SwapX — CEX + DEX Crypto Exchange

2026 Full-Stack Web3 Mechanics Live: easygoing-motivation-production.up.railway.app · Repo: github.com/expediator/swapx-dex

WHAT IT IS

A full-stack crypto trading platform combining two market structures: a **CEX (Centralised Exchange)** with a traditional order-book, and a **DEX (Decentralised Exchange)** with an Automated Market Maker (AMM). Users trade 30+ tokens at live CoinGecko prices, with simulated portfolios stored per-user in Upstash Redis. Google OAuth handles identity.

ARCHITECTURE

Layer	Technology	Role
Frontend	Next.js 14 App Router + React 18	Server components, client UI, routing, streaming
Styling	Tailwind CSS + shadcn/ui + Lucide React	Utility classes, accessible components, SVG icons
Charts	Recharts	Portfolio value line chart, price history
Auth	NextAuth.js v4 (Google OAuth 2.0)	User sessions; per-user Redis key isolation
Validation	Zod	Runtime schema validation for all API route inputs
Cache / DB	Upstash Redis (REST)	Price cache (60s TTL), portfolio hashes, trade history lists
Prices	CoinGecko API	Live USD prices for 30+ tokens on free tier
Deploy	Railway	PaaS; `railway.toml` config; sleeps after 15 min idle on free tier

CEX — ORDER-BOOK MECHANICS

The CEX mode uses an **order-book** — the mechanism behind Binance, Coinbase, Kraken. Two order types:

- **Market Order** — executes immediately at the current CoinGecko price. Fee: 0.1% of trade value.
- **Limit Order** — user sets a target price; fills only when the market price crosses that level on the next poll.

DEX — AMM (X × Y = K)

The DEX uses the **constant-product formula** — same mechanic as Uniswap v2. A simulated liquidity pool holds reserves of two tokens. Their product must remain constant after every swap:

```
Pool: 1000 ETH · 2,000,000 USDC → k = 2,000,000,000

Swap 10 ETH (0.3% fee deducted first → 9.97 ETH enters):
  new_ETH  = 1000 + 9.97 = 1009.97
  new_USDC = k / 1009.97  = 1,980,217
  received = 2,000,000 - 1,980,217 = 19,783 USDC
  price    = 19,783 / 10  = 1,978.3 USD/ETH  (vs 2,000 market → 1.1% slippage)
```

Larger swaps relative to pool size = higher slippage. SwapX shows the estimated slippage before confirming.

REDIS DATA MODEL

Key Pattern	Type	What's Stored
`prices:<token>`	String + TTL 60s	CoinGecko USD price, auto-expires every minute
`portfolio:<userId>`	Hash	token → balance mapping per authenticated user
`trades:<userId>`	List	Trade history (JSON strings), newest first via LPUSH

`orders:<userId>`	List	Open limit orders awaiting fill

AUTHENTICATION FLOW (OAUTH 2.0 VIA NEXTAUTH)

1. User clicks "Sign in with Google" → redirected to `accounts.google.com/o/oauth2/auth` with `client_id`, `scope`, `redirect_uri`
2. Google authenticates + asks consent, then returns a short-lived `code` to `redirect_uri`
3. NextAuth exchanges `code` for `access_token` + `id_token` (JWT containing user email/name/picture)
4. NextAuth creates a session JWT signed with `NEXTAUTH_SECRET`, stored in an HTTP-only cookie
5. Every API route calls `getServerSession()` to verify the cookie and get the user ID for Redis key isolation

Hand Gesture & Voice Controlled RC Car

2025 Hardware CV Embedded Repo: github.com/expediator/gesture-controlled-robot

WHAT IT IS

A physical RC car controlled via two simultaneous input modes: real-time **hand gesture recognition** (webcam + MediaPipe) and **offline voice commands** in English + Hindi (Vosk STT). Commands travel from a Python script over **UDP** on local WiFi to an ESP32 microcontroller driving the motors. Built solo in 4 weeks.

SYSTEM ARCHITECTURE

Component	Technology	Role
Vision input	OpenCV + MediaPipe Hands (Python)	Webcam capture; 21 hand landmark detection per frame
Voice input	Vosk STT (offline)	Mic → text in English/Hindi; ~50ms latency; no internet
Command routing	Python UDP socket	Sends directional strings (F/B/L/R/S) to ESP32 on LAN
Microcontroller	ESP32 (Arduino/Embedded C)	Receives UDP packets; drives motor driver via PWM
Motor driver	L298N H-Bridge	Controls 2 DC motors independently (forward/reverse per motor)
Obstacle sensor	HC-SR04 ultrasonic	Measures distance; auto-halts car if obstacle <15 cm
Power	LiPo battery (7.4V 2S, 20C+)	High-discharge for motors; 5V regulator for ESP32 logic
Math	NumPy	Landmark vector angles (dot product → arccos) for gesture classification

GESTURE CLASSIFICATION

MediaPipe detects 21 hand landmarks (x, y, z) per frame. NumPy computes angles between finger-bone vectors to classify gestures:

- **Open palm** → Forward | **Fist** → Stop
- **Wrist tilted left/right** (roll angle) → Turn left / right
- **Thumb pointing down** → Reverse

WHY VOSK (OFFLINE) VS CLOUD STT

	Vosk (offline)	Google Cloud STT
Latency	~50 ms (local inference)	300–800 ms (round-trip)
Internet required	No	Yes

API cost	Free	$0.006/15 sec
Model size	~50 MB (EN), ~35 MB (HI)	Served remotely
Verdict for RC car	**Required** — real-time control needs <100ms	Too slow

WHY UDP OVER TCP FOR CONTROL

UDP is fire-and-forget — no handshake, no ACK wait, no re-transmission. For streaming control signals (20–30 commands/sec), a dropped UDP packet causes a brief hesitation. A TCP re-transmission would deliver a stale "turn left" command 200ms late — causing erratic, dangerous movement. UDP's "unreliability" is actually the safer choice for real-time robot control.

FaceMetrics — AI Facial Geometry Analysis

2026 CV Browser ML Live: expediator.github.io/face-rater · Repo: github.com/expediator/face-rater

WHAT IT IS

A browser-based facial analysis tool — 100% client-side, no server, no data upload. Uses MediaPipe FaceMesh's 468 landmarks to score 11 facial geometry metrics from webcam input. Evolved through **16 iterations**: Python CLI → Tkinter GUI → OpenCV app → final JavaScript browser version.

THE 11 FACIAL METRICS

Metric	Landmarks Used	Calculation
Jaw Width Ratio	Jaw corners	Jaw width / cheekbone width
Cheekbone Ratio	Cheek landmarks 234, 454	Cheekbone width / jaw width
Facial Symmetry	All bilateral pairs	Mean absolute L/R landmark deviation
Eye Area	Eye contour landmarks	Pixel area of eye aperture from Canvas polygon
Canthal Tilt	Inner/outer eye corners	Angle of line between outer vs inner canthus
Chin Projection	Chin tip, jaw base	Chin-to-jaw depth / face height ratio
Nose-Lip Distance	Nose base, upper lip	Pixel distance normalized to face height
Interpupillary Dist.	Pupil centers	IPD / face width
Forehead Ratio	Hairline, brow	Forehead height / total face height
Lip Fullness	Upper/lower lip contours	Lip height / lip width from landmarks
Dimorphism Score	All metrics combined	Weighted composite; weight table differs by declared sex

HOW MEDIAPIPE WASM RUNS IN-BROWSER

MediaPipe compiles its ML inference engine to **WebAssembly** — a binary format that V8 runs at near-native speed. The model file loads once (~8 MB), then all inference (468 landmark coordinates per frame) happens locally. JavaScript reads specific landmark indices and uses the Canvas 2D context to draw overlays and compute distances/angles. Zero data leaves the device.

Space Shooter — 2D Arcade Game

2025 Game Dev Canvas API Live: expediator.github.io/space-shooter · Repo: github.com/expediator/space-shooter

WHAT IT IS

A 2D arcade space shooter originally written in Python (pygame), then completely rewritten in pure JavaScript using the HTML5 Canvas API — runs in any browser instantly with no install.

Aspect	Python / pygame	JavaScript / Canvas
Distribution	Requires Python + pygame installed	URL — shareable to anyone instantly
Game loop	`clock.tick(60)`	`requestAnimationFrame(loop)`
Drawing	`pygame.draw.rect(surface, color, rect)`	`ctx.fillRect(x, y, w, h)`
Input	`pygame.event.get()`	`addEventListener('keydown', …)`
Physics	Manual velocity vectors	Same — velocity + drag each frame

- **Thrust + Inertia Physics** — velocity accumulates with thrust, decays with drag coefficient each frame
- **Two AI Enemy Types** — one chases player; one leads shots by predicting player position
- **Particle System** — explosion pool with velocity, opacity decay, color per particle
- **Power-ups** — shield, rapid-fire, spread-shot spawn at intervals
- **Touch Support** — virtual joystick for mobile browsers via touch events

Interactive Portfolio — Windows OS Style

2025 Frontend Vanilla JS Live: expediator.github.io/resume

WHAT IT IS

An interactive portfolio site that mimics a Windows desktop — draggable windows, taskbar, desktop icons, music player. Pure HTML + CSS + vanilla JavaScript. Zero frameworks. Hosted on GitHub Pages.

Feature	Implementation
Draggable windows	`mousedown` → track `mousemove` delta; update `left/top` CSS; z-index bump on focus
Music player	Spotify embed iframes inside the Music Library window — no self-hosted audio (copyright compliance)
GitHub heatmap	Embedded via `ghchart.rshah.org/expediator` — renders the green contribution graph as an SVG
Rotating wallpapers	CSS background transitions on a JS timer + a wallpaper picker UI
Windows (Skills/Projects/Contact)	Each is a draggable window acting as a section of the resume
Cache-busting	`script.js?v=7` + `style.css?v=7` — version bumped on every JS/CSS edit to force browser refresh

03

Tech Stack Applied — Where Every Tool Was Used

LANGUAGES

Language	Version	Projects Applied In	What For
TypeScript	5.x	SwapX	All pages, API routes, components — full type safety end-to-end
JavaScript	ES2024	FaceMetrics, Space Shooter, Portfolio	Canvas drawing, MediaPipe integration, game loop, UI logic
Python	3.12	RC Car, FaceMetrics (early)	CV pipeline, Vosk STT, NumPy math, Tkinter GUI
C (Embedded)	C11	RC Car (ESP32 firmware)	GPIO, PWM, UDP socket, motor control via Arduino framework
C++	C++17	Study (systems, game dev)	Classes, templates, smart pointers, STL algorithms; used in CUDA/OpenGL guide projects
SQL	—	Study	Relational DB concepts; Redis chosen for SwapX for lower latency
Solidity	0.8.x	Web3 study	Smart contract basics; ERC-20 patterns; storage vs memory
Rust	1.78	Study (learning)	Memory ownership model; Anchor Framework for Solana programs
Bash	—	DevOps / scripts	Build automation, Chrome headless PDF generation, git scripting

FRAMEWORKS & RUNTIME

Tool	Applied In	What For
Next.js 14 App Router	SwapX	SSR, API routes as serverless functions, middleware, dynamic routing, streaming
React 18	SwapX	Component UI; hooks: `useState`, `useEffect`, `useCallback`, `useTransition`
Node.js 22	SwapX (runtime)	Server runtime; native `fetch`, `crypto` APIs; ESM support
Tailwind CSS	SwapX	Utility-first styling; JIT compiler outputs only used classes
Express.js	Prototyping / study	REST API prototypes; middleware chains before adopting Next.js API routes
Prisma ORM	Study	TypeScript-first ORM for PostgreSQL/SQLite; auto-generated client from schema; migrations; integrates tightly with Next.js

AUTH, DATA & VALIDATION

Tool	Applied In	What For
NextAuth.js v4	SwapX	Google OAuth; session management; CSRF protection built-in
OAuth 2.0	SwapX	Authorization code flow via Google; app never sees the user's password
JWT	SwapX	Session token format; header.payload.signature; signed with `NEXTAUTH_SECRET`
Zod	SwapX	Runtime API input validation; schema also infers TS types — single source of truth
Upstash Redis	SwapX	HTTP-based Redis; price TTL cache + portfolio hashes + trade history lists

REST API Design	SwapX	Resource-oriented routes: `POST /api/trade`, `GET /api/prices/:token`
WebSockets	Study	Full-duplex TCP for real-time price streaming; planned for live ticker feature
PostgreSQL	Study	Relational DB; ACID transactions; parameterised queries (`$1, $2`); indexes, triggers; used in all guide projects

UI & VISUALIZATION

Tool	Applied In	What For
Recharts	SwapX	Portfolio value line chart, token price history bar charts; declarative React API
Lucide React	SwapX	Tree-shakeable SVG icon library (~1,400 icons); wallet, chart, trade icons
Canvas API	FaceMetrics, Space Shooter	FaceMetrics: landmark overlay drawing; Space Shooter: full game rendering
shadcn/ui	SwapX	Copy-paste Radix UI + Tailwind components (Dialog, Button, Input) — no package dep
CoinGecko API	SwapX	Free REST API; `/simple/price?ids=<token>&vs_currencies=usd` ; ~10–30 req/min free

CV, ML & EMBEDDED

Tool	Applied In	What For
OpenCV	RC Car, FaceMetrics (Python)	Webcam capture (`VideoCapture`), frame processing, brightness normalization
MediaPipe FaceMesh	FaceMetrics	468 facial landmarks per frame; WASM in browser + Python native version
MediaPipe Hands	RC Car	21 hand landmarks per frame for gesture classification
Vosk STT	RC Car	Offline STT; EN + HI models; ~50ms latency; Kaldi-based; free
NumPy	RC Car, FaceMetrics	Landmark array math; dot products; arccos for angles; batch distance calc
pandas	Trading Sim / study	OHLCV data manipulation; rolling averages; strategy backtesting
Arduino/ESP32	RC Car	ESP32: 240MHz dual-core, built-in WiFi; Arduino HAL over hardware registers
Embedded C	RC Car	Low-level ESP32 firmware: `digitalWrite`, `analogWrite` PWM, `WiFiUDP`, ISRs
scikit-learn	Study	Classical ML algorithms (linear regression, SVM, decision trees, k-NN); `fit()` / `predict()` API; pairs with NumPy + pandas
Matplotlib	Study / Trading Sim	Python plotting library; line/bar/scatter charts; `plt.plot()`, `plt.show()` ; pairs with pandas for data analysis

DEVOPS & INFRASTRUCTURE

Tool	Applied In	What For
Git	All projects	Version control; branching; commit history across all 7+ repos
GitHub	All projects	Remote hosting; GitHub Pages deployment; repo management
Railway	SwapX	PaaS auto-deploy from git; `railway.toml` ; free tier sleeps after 15 min idle
GitHub Pages	FaceMetrics, Space Shooter, Portfolio, Tic-Tac-Toe	Free static hosting; deploys from `main` branch root automatically
GitHub Actions	Learning	CI/CD YAML workflows; trigger on push; run tests + auto-deploy

Docker	Learning	Containerisation; Dockerfile; images run identically in dev/staging/prod
Linux / Ubuntu	Server environments	CLI navigation, bash scripting, permissions, SSH, package management (`apt`); Docker container base images; WSL on Windows

WEB3 (ACTIVELY EXPLORING)

Tool	Applied In	What For
Solana	Study	High-throughput L1 (~65K TPS); Proof of History; account model (not UTXO)
Ethereum	Study	EVM; gas mechanics; Proof of Stake (post-Merge 2022); smart contract state
Solidity	Study	EVM bytecode language; `mapping(address ⇒ uint256)` ; ERC-20; events
ethers.js	Study	JS library to connect wallet, read contract state, send txns on Ethereum
Anchor Framework	Study	Rust framework for Solana programs; IDL generation; Borsh serialization
DeFi / AMM	SwapX (simulated)	Implemented x*y=k; slippage, liquidity depth, fee mechanics — fully understood

AI & LLMS (ACTIVELY EXPLORING)

Tool	Applied In	What For
PyTorch	Study	Tensor operations, autograd engine, training loop: `zero_grad() → forward() → loss → backward() → step()`
HuggingFace Transformers	Study	Pre-trained LLMs and tokenizers; `AutoTokenizer` , `AutoModel` , `pipeline()` API for classification, generation, embeddings
LangChain	Study	LLM application framework; chains, agents, tools, memory; ReAct agent loop (Thought → Action → Observation)
FastAPI	Study	Python async REST API for serving ML models; OpenAPI docs auto-generated; Redis caching via `setex()`
ChromaDB	Study	Vector database; `PersistentClient` ; stores sentence embeddings; cosine similarity retrieval for RAG
Ollama	Study	Run LLMs locally (Llama 3, Mistral, Phi); `generate()` API; used as RAG generation backbone
RAG Pipelines	Study	Retrieval-Augmented Generation: chunk → embed → store → retrieve → generate; reduces LLM hallucinations

04

Core Concepts Explained

1 · CEX vs DEX — Structural Difference

Aspect	CEX (Order-Book)	DEX (AMM)
Price discovery	Buyers/sellers post limit orders; price = where orders match	Algorithmic: ratio of two token reserves determines price
Custody	Exchange holds user funds	Smart contract holds liquidity; user keeps wallet keys
Liquidity source	Market makers posting bid/ask orders	Liquidity providers deposit token pairs into the pool
Slippage	Minimal on deep order books	Always present; proportion of trade to pool size
Real examples	Binance, Coinbase, Kraken	Uniswap, Jupiter (Solana), Raydium, Curve
SwapX implementation	Market + limit orders, 0.1% fee	x*y=k formula, 0.3% fee, slippage preview shown

2 · Constant-Product AMM — x × y = k

```
Pool holds: reserve_A (token A) and reserve_B (token B)
Invariant: reserve_A × reserve_B = k  (constant, never changes)

Swap Δa of token A in (after 0.3% fee: effective_Δa = Δa × 0.997):
  new_A = reserve_A + effective_Δa
  new_B = k / new_A
  received_B = reserve_B - new_B

Price impact grows non-linearly as trade size → pool size.
This is slippage. A 10% of pool trade causes ~10% price movement.
```

3 · OAuth 2.0 — Authorization Code Flow

1. App redirects user to Google: `accounts.google.com/o/oauth2/auth?client_id=…&redirect_uri=…&scope=openid+email+profile&response_type=code`
2. Google authenticates + user consents → Google redirects back with a short-lived `?code=…`
3. Server POSTs `code` + `client_secret` to `/token` endpoint → receives `access_token` + `id_token`
4. `id_token` is a signed JWT containing: sub (user ID), email, name, picture
5. NextAuth stores a session JWT in an HTTP-only cookie (signed with `NEXTAUTH_SECRET`)
6. Every API call → server reads cookie → verifies signature → gets userId → reads/writes Redis

> The app never sees the user's Google password. OAuth delegates authentication to Google; the app only receives a verified identity token.

4 · Redis Caching Strategy (Read-Through + TTL)

CoinGecko free tier allows ~10–30 req/min. Without caching, concurrent users would instantly exhaust this limit.

```
function getPrice(token):
  cached = redis.get("prices:" + token)
  if cached: return cached          // sub-ms Redis read

  price = coingecko.fetch(token)    // only if cache miss
```

```
  redis.set("prices:" + token, price, EX=60)   // 60s TTL
  return price
```

After 60 seconds the key expires automatically, and the next request fetches fresh data. This is **read-through cache with TTL expiry**.

5 · JWT — How Session Tokens Work

```
Structure: base64url(header) . base64url(payload) . signature

header:  { "alg": "HS256", "typ": "JWT" }
payload: { "sub": "userId123", "email": "user@example.com", "iat": 1718000000, "exp": 1718086400 }
signature: HMAC-SHA256(header + "." + payload, NEXTAUTH_SECRET)
```

The server verifies the signature on every request — no database lookup needed. If the secret is correct and the token isn't expired, the user is authenticated. Stateless.

6 · Zod — Runtime + Compile-Time Validation

TypeScript types are erased at runtime. A `number`-typed parameter gets no protection from a client sending `"hello"`. Zod bridges the gap:

```
const TradeSchema = z.object({
  token:  z.string().min(1),
  amount: z.number().positive().max(1_000_000),
  type:   z.enum(["buy", "sell"]),
  order:  z.enum(["market", "limit"]),
  limit:  z.number().positive().optional(),
});

type TradeInput = z.infer<typeof TradeSchema>;  // TypeScript type, free

const result = TradeSchema.safeParse(req.body);
if (!result.success) return res.status(400).json({ errors: result.error.flatten() });
```

7 · MediaPipe — WASM in Browser

MediaPipe compiles its C++ inference engine to **WebAssembly**. WASM is a binary instruction format V8 (Chrome's JS engine) executes at near-native speed — no install, no server. The FaceMesh model (~8 MB) loads once; then each video frame is passed to the WASM function which returns 468 `(x, y, z)` landmark coordinates normalized to [0, 1]. JavaScript reads specific indices (e.g., 234 = left cheek, 454 = right cheek) and uses the Canvas 2D context for geometric calculations and overlays.

8 · UDP vs TCP for Robotics Control

Property	TCP	UDP	RC Car verdict
Delivery guarantee	Yes — retransmits on drop	No — fire and forget	UDP preferred
Ordering	Guaranteed	Not guaranteed	UDP preferred
Latency	Higher (SYN/ACK + retransmit)	Lower (no overhead)	UDP preferred
Stale command risk	High — old "turn" retransmitted late	Low — new command arrives instead	UDP preferred

For streaming control signals at 20–30 Hz, a dropped frame = brief hesitation. A late TCP retransmission = executing a stale direction command = dangerous. UDP wins for real-time robotics.

05

Skills Reference by Category

What each technology actually does — useful for interviews and self-review.

LANGUAGES

Skill	What It Is & Key Facts
TypeScript	JS superset with static types; erased at build time. Catches type errors before runtime. Structural typing — if it has the right shape, it's the right type.
JavaScript ES2024	Features used: async/await, Promise.all, optional chaining (`?.`), nullish coalescing (`??`), structuredClone, Array groupBy.
Python 3.12	Used for CV and hardware scripts. 3.12 improves error messages, f-string flexibility, and adds `typing.override` . GIL still present (3.13 removes it).
C (Embedded)	No GC, no runtime overhead. On ESP32 via Arduino HAL: `pinMode` , `digitalWrite` , `analogWrite` (PWM duty cycle), `Serial.print` , interrupt handlers.
SQL	SELECT, JOIN, WHERE, GROUP BY, INDEX, transactions (ACID). Conceptual for SwapX — Redis chosen for <1ms latency over SQL's disk I/O.
Solidity	Compiled to EVM bytecode. Key: `mapping(address ⇒ uint)` for state, `event` for off-chain indexing, `modifier` for access control, `payable` for ETH receive.
Rust (learning)	Ownership model: every value has one owner; borrow checker enforces no data races at compile time. Used in Anchor Framework for Solana programs.
Bash	Shell scripting: pipes, conditionals, loops, environment variables. Used for build automation and Chrome headless PDF generation scripts.

FRAMEWORKS & RUNTIME

Skill	What It Is & Key Facts
Next.js 14 App Router	React meta-framework. App Router: React Server Components (run on server, zero JS sent to client), nested layouts, streaming with Suspense, API routes as serverless functions under `/app/api/` .
React 18	UI library. 18 adds concurrent rendering (`useTransition`), automatic batching, `Suspense` for data fetching, and `useId` . SwapX uses hooks extensively for trade form state.
Node.js 22	V8-based server JS runtime. LTS release: built-in `fetch` , Web Streams API, improved ESM support, built-in test runner. SwapX runs on Node 22 on Railway.
Tailwind CSS	Utility-first CSS. Classes like `flex items-center gap-4 rounded-lg bg-blue-600 hover:bg-blue-700` . JIT compiler scans HTML/JSX and outputs only classes that exist in the markup.
Express.js	Minimal Node.js web framework. Middleware pipeline: `app.use(middleware)` chains transform `req/res` . Used for REST API prototypes before Next.js API routes.

AUTH, DATA & VALIDATION

Skill	What It Is & Key Facts
NextAuth.js v4	Auth library for Next.js. Handles OAuth flows, session JWT signing/verification, CSRF tokens, and secure cookie management. Config: `NEXTAUTH_SECRET` + `NEXTAUTH_URL` .
OAuth 2.0	Authorization protocol (not auth itself). Delegates identity to a provider. Four flows: Authorization Code (used in SwapX), Implicit (deprecated), Client Credentials, Device Code.
JWT	3-part token: `header.payload.sig` . Stateless: server verifies signature without DB hit. Never store sensitive data in payload — it's base64 encoded, not encrypted.

Zod	Schema validation + TS type inference from a single schema definition. `z.infer<typeof Schema>` gives the TypeScript type. `safeParse` returns success/error without throwing.
Upstash Redis	Serverless Redis over HTTP REST. No persistent TCP — works in any environment. Commands: `GET`, `SET`, `HSET`, `HGET`, `LPUSH`, `LRANGE`. Free: 10K req/day.
REST API Design	Stateless HTTP: nouns in URLs (`/api/trades`), verbs via HTTP methods (GET read, POST create, PUT update, DELETE remove), JSON bodies, proper status codes (200/201/400/401/500).
WebSockets	HTTP upgrade to full-duplex TCP. Client and server can push messages at any time — no polling. Used for real-time price tickers, live order books, chat. Protocol: `ws://` or `wss://`.

UI & VISUALIZATION

Skill	What It Is & Key Facts
Recharts	React chart library built on D3. Declarative: `<LineChart><Line dataKey="value"/></LineChart>`. Responsive via `<ResponsiveContainer>`. Used for portfolio value line chart in SwapX.
Lucide React	~1,400 SVG icons, MIT license, tree-shakeable. Import only what you use: `import { Wallet, TrendingUp } from 'lucide-react'`.
Canvas API	Browser 2D drawing context. Imperative: clear → draw each frame. Used for landmark overlay polygon drawing (FaceMetrics) and full game rendering (Space Shooter) at 60fps via `requestAnimationFrame`.
shadcn/ui	Not a package — it's a CLI that copies component source code into your project. Built on Radix UI primitives (accessible, unstyled) + Tailwind. No version dependency after copy.
CoinGecko API	Free crypto data REST API. `GET /simple/price?ids=bitcoin,ethereum&vs_currencies=usd`. Rate limit ~10–30 req/min free tier. Used with 60s Redis cache in SwapX to avoid hitting limits.

CV, ML & EMBEDDED

Skill	What It Is & Key Facts
OpenCV	C++ CV library with Python bindings. Key functions: `VideoCapture(0)` for webcam, `cvtColor(BGR→RGB)`, `equalizeHist` for brightness, `imshow` for display.
MediaPipe FaceMesh	Google ML model; 468 (x,y,z) landmarks normalized to [0,1]. Runs as WASM in browser (>30fps) or native Python. Each landmark index has a fixed semantic meaning across all faces.
Vosk STT	Offline speech recognition based on Kaldi. Python: reads mic audio in a thread, passes chunks to `KaldiRecognizer`, parses JSON result. No internet. EN model: ~50MB, HI model: ~35MB.
NumPy	N-dim array math. Used for: converting landmark lists to arrays, computing Euclidean distances (`np.linalg.norm`), dot products for angles (`np.arccos(np.dot(u,v))`), batch operations.
pandas	DataFrame-based data analysis. Key: `df.rolling(N).mean()` for moving averages, `df.groupby()`, vectorized operations. Used in trading strategy simulator on OHLCV CSV data.
Arduino/ESP32	ESP32 = dual-core 240MHz Xtensa LX6 + WiFi + BT. Arduino framework abstracts hardware to `setup()` + `loop()`. PWM: `ledcWrite(channel, duty)` for motor speed control.
Embedded C	C without OS; direct hardware registers. On ESP32: `WiFi.begin(ssid, pass)` → `udp.begin(port)` → `udp.read(buf, len)` → parse command → `digitalWrite(motorPin, HIGH)`.

DEVOPS & INFRASTRUCTURE

Skill	What It Is & Key Facts
Git	Distributed VCS. Key commands used: `git add -p` (patch staging), `git stash`, `git rebase`, `git log --oneline --graph`. All 7+ project repos maintained.
GitHub	Git hosting + developer platform. Uses: repo hosting, GitHub Pages (static deploy), Issues, Pull Requests, GitHub Actions CI/CD, profile README (expediator/expediator).
Railway	PaaS (Platform as a Service). Auto-detects Next.js; builds with `npm run build`; serves with `npm run start`. Configured via `railway.toml`. Free tier: 500 compute hours/month; sleeps after 15 min idle.

GitHub Pages	Free static hosting; serves directly from a git branch (typically `main`). Supports custom domains. URL format: `username.github.io/repo`. No build step — serves HTML/CSS/JS as-is.
GitHub Actions	CI/CD built into GitHub. Workflow = YAML file in `.github/workflows/`. Trigger on `push` or `pull_request`. Steps run in a container: install → test → build → deploy.
Docker	Containerisation: app + dependencies packed into an image. Dockerfile defines layers (each instruction = layer, cached). `docker run -p 3000:3000 myapp` runs the container. Same image in dev/staging/prod.

WEB3 (ACTIVELY EXPLORING)

Skill	What It Is & Key Facts
Solana	L1 blockchain; ~65K TPS via parallel transaction processing. Proof of History (verifiable time ordering) + Tower BFT consensus. Programs are stateless; state lives in separate accounts.
Ethereum	EVM blockchain. Smart contracts store state on-chain. Gas = compute cost in Gwei. Proof of Stake since Sep 2022 (The Merge). Solidity most common contract language.
Solidity	High-level language → EVM bytecode. Key: `mapping` (hash map on storage), `event` (log for off-chain indexing), `modifier` (function guards), `view` / `pure` (no state change).
ethers.js	JS/TS library for Ethereum. `new ethers.BrowserProvider(window.ethereum)` → connect MetaMask. `contract.read()` / `contract.write()`. Handles ABI encoding/decoding automatically.
Anchor Framework	Rust framework for Solana programs. `#[program]` macro generates IDL (like ABI). Enforces account validation via `#[account]` constraints. `anchor test` runs TypeScript test suite.
DeFi / AMM	Financial primitives on-chain. AMM replaces order-books: algorithmic price = f(reserves). Key variants: constant-product (Uniswap), stableswap (Curve), concentrated liquidity (Uniswap v3).

All skills marked (learning) — Rust, Docker, GitHub Actions, GitHub Actions, Anchor, Solana/Ethereum in-depth — are actively being studied and are not yet in production projects. All other tools have direct application in at least one shipped project above.

Resume PDF: expediator.github.io/resume/assets/Akshansh-Yadav-Resume.pdf
Interactive Portfolio: expediator.github.io/resume
GitHub: github.com/expediator
SwapX (live): easygoing-motivation-production.up.railway.app